

研究背景

动态可搜索加密 (Dynamic Searchable Encryption, DSE) 允许客户端在云端维护加密数据库的同时, 支持对数据的增删查操作。在早期的 DSE 方案中, 尽管支持了高效检索, 却容易被“文件注入攻击”关联新更新与历史查询, 因而提出了**前向隐私**与**后向隐私**的安全目标: 前向隐私保证服务器无法将新更新与此前的同关键词查询关联; 后向隐私保证删除后的文档不再被后续搜索泄露。多数高效 DSE 方案在实现这两种隐私时, 都需要为每个关键词维护一个可更新的本地状态 (如计数器), 导致客户端存储开销与关键词数线性相关, 不利于关键词丰富的场景部署。

之前研究的不足

1. **客户端存储线性增长**: 基于计数器的方案 (如Sophos、FAST、Mitra等) 需为每个关键词保存状态, 存储开销 $O(|W|)$;
2. **ORAM 方案效率低**: 如Moneta、Orion、Horus 等通过 ORAM/OMAP 外包状态, 虽降低客户端存储, 但通信与轮次开销高, 实际效率欠佳;
3. **CLOSE-FB 的可用性瓶颈**: 采用固定长度哈希链实现全局计数器, 仅需常数客户端存储, 但当“链用尽” (计数器达到预设上界 L) 时, 必须整库重构, 严重影响可用性与性能。

核心创新点

1. **DISCO 框架**: 提出基于**范围受限伪随机函数** (RCPRF) 和全局批次计数器的通用 DSE 设计框架, 实现前向&后向隐私同时仅需常数客户端状态;
2. **RCPRF 属性抽象**: 形式化并分析了三类 RCPRF (哈希链、GGM 树、Trapdoor Permutation), 归纳出关键性能与可用性属性, 并提出混合 RCPRF 设计以兼顾多重优势;
3. **可配置实例化**: 在 DISCO 框架下, 给出三种具体构造—— DISCO_h (哈希链)、 DISCO_g (GGM 树)、 DISCO_t (TDP), 在计算、通信、支持域大小上提供多种折中;
4. DISCO^+ **优化**: 通过并行搜索、搜索后重键 (post-search re-keying) 和批库合并等技术, 进一步降低服务器端检索及后续 RCPRF 评估开销。

相比前人方案的优势

- **常数客户端存储**: 仅需存储全局计数器与对称密钥, 总量 $\leq 2\text{KB}$; 而传统方案需数百 MB 用于关键词状态。
- **通信效率不降**: 更新/搜索通信量与最优轻量级方案 (Sophos/FAST/Mitra) 相当, 均为 $O(a_w)$; DISCO_g 额外只需发送 $O(\log L)$ 大小的受限密钥。
- **实用性能优越**: 实验表明, DISCO_g 在 LAN/WAN 场景下更新与检索延迟均在毫秒级, 且对比非常数存储的 DSE 方案, 仅引入“人感知不到”的微弱计算开销。
- **灵活性强**: 可根据应用需求在三种构造间选取, 或采用混合 RCPRF 来兼具高效与无限域支持, 避免 CLOSE-FB 重建痛点。

方案实现详述

1. 初始化 Setup

- 客户端生成两把密钥: PRF 密钥 k_1 (用于派生关键词专属 RCPRF 主密钥)、对称加密密钥 k_2 ; 初始化全局批次计数器 $c \leftarrow 0$ 。

2. 批次更新 Update

- 客户端 $c \leftarrow c + 1$; 对于每个新出现关键词 w , 用 k_1 和 w 计算 RCPRF 主密钥 $k_r^{(w)}$; 再对批次编号 c 生成受限密钥计算出搜索令牌 st_w^c 。
- 对该批次内 w 的每个更新操作伪随机函数 $u = H_1(st_w^c | j)$ (j 为子计数) 作为索引, 加密元组 (op, id) 后上传。

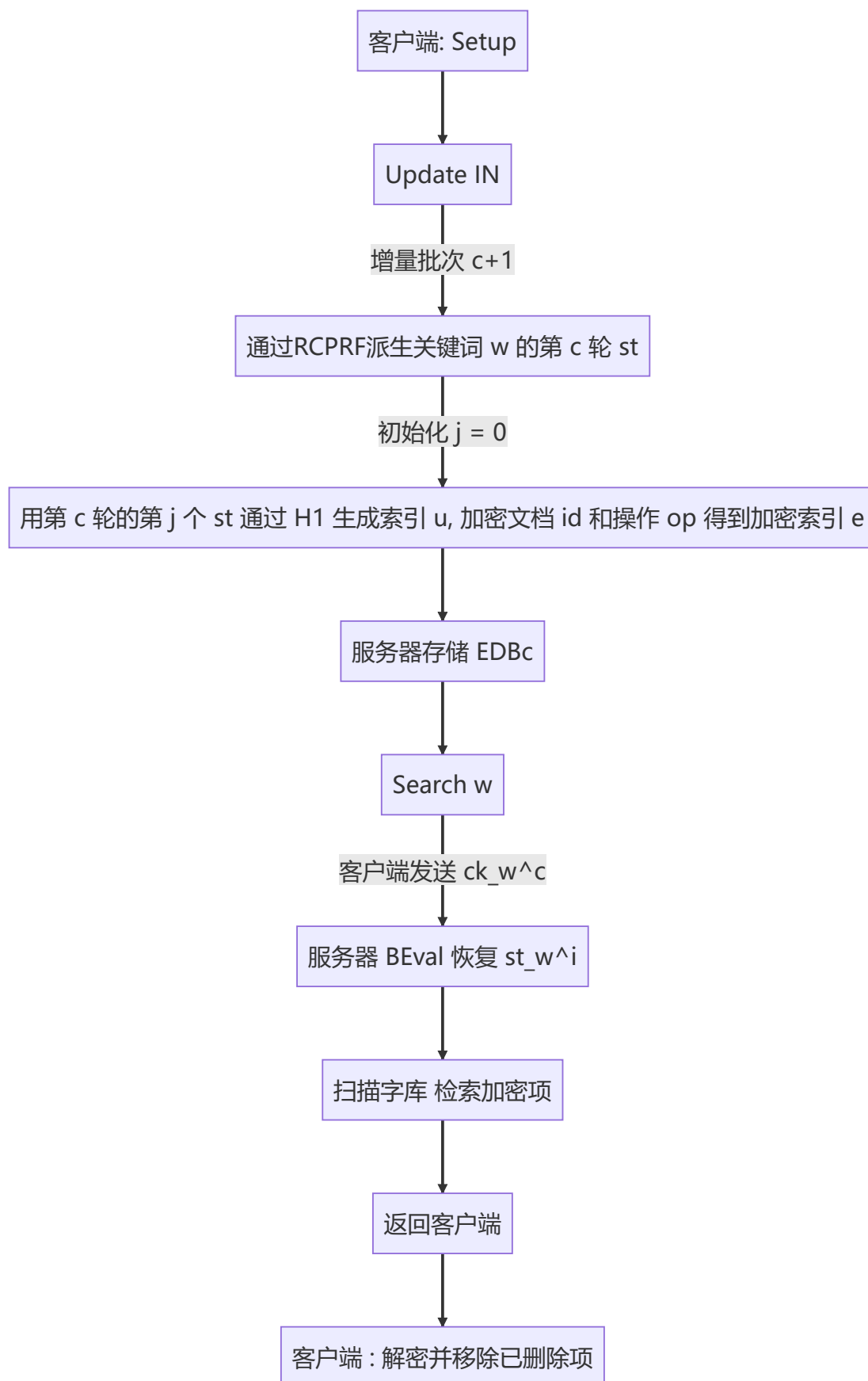
3. 关键词检索 Search

- 客户端派生受限密钥 ck_w^c 并发送给服务器; 服务器利用 RCPRF.BEval 恢复 $st * w^i * i = 1^c$, 在每个子数据库扫描对应 $u = H_1(st_w^i | j)$ 取回加密项集合, 再返还给客户端解密、合并并剔除已删除项。

4. DISCO+ 优化

- **并行搜索**: 在加密项中嵌入子计数使服务器预先知晓匹配数, 动态分配线程并行 I/O / 令牌生成。
- **后搜索重键**: 服务器用最新令牌对已检索项重新加密并更新数据库, 从而下次搜索无需再评估历史 RCPRF。
- **数据库合批**: 客户端按需下载若干子库, 合并重加密后上传, 减少全局计数器扫描子库次数。

流程示意



主要结果与结论

- **更新性能:** 在 $\alpha = c/L \in [0.2, 0.8]$ 范围内, $DISCO_g$ 更新时延恒定约 0.01,ms/对, $DISCO_h$ 随 α 增大而效率提升, $DISCO_t$ 在大 L 小 α 时优于 $DISCO_h$;

- **搜索性能**：DISCO_g 得益于并行优化，在大规模匹配场景下实现业界领先的检索速度，且通信开销仍与轻量级方案一致；DISCO_h 在 $\alpha \rightarrow 1$ 时次优，DISCO_t 支持无上界域但计算稍高；
- **存储开销**：客户端仅需常数 $< 2\text{KB}$ ，服务器端数据库大小与传统方案持平；相比需数百 MB 客户端存储的方案优势显著。
- **安全性**：形式化证明 DISCO 满足前向隐私与 II 型后向隐私，DISCO⁺ 的优化不额外增加泄露。

总体而言，DISCO 在保持或接近最优通信与轮次的同时，实现了常数客户端存储与高效可用的动态可搜索加密，是小客户端存储 DSE 领域的一大突破。

范围受限的伪随机函数

$k \leftarrow F_{rc}.KeyGen(1^\lambda)$

这段话的核心意思是：在基于哈希链（hash-chain）和基于陷门置换（TDP）的 RCPRF 构造里，作者不仅仅把内部链上的值 s_i （或者置换后的值 $\pi_{pk}^{-1}(s_i)$ ）直接当作“可约束 PRF”在第 i 个位置的输出（记作 st_i ），而是额外对它们再打一层单向哈希 H ，即

$$st_i = H(s_i) \quad (\text{或 } H(\pi_{pk}^{-1}(s_i))) .$$

为什么要多这一步？关键在于**安全定义**里要求——即使对手拿到了“第 c 个批次”的约束密钥 $OConstrain_k([c])$ ，也绝对不能分辨真正的 RCPRF 输出和随机数的区别。

- 如果把输出直接设成 s_i （或 $\pi_{pk}^{-1}(s_i)$ ），那么当对手：
 1. 查询了“约束到前 c 批次”的密钥，就得到了内部状态 s_c 。
 2. 然后对第 $c+1$ 批次发起挑战查询，就会拿到一个输出值 y 。

由于哈希链（或陷门置换）本身就是一个可预测 / 可逆的关系——哈希链里 $s_{c+1} = g(s_c)$ ，陷门置换里 $\pi_{pk}(y) = s_c$ ——对手只要检查

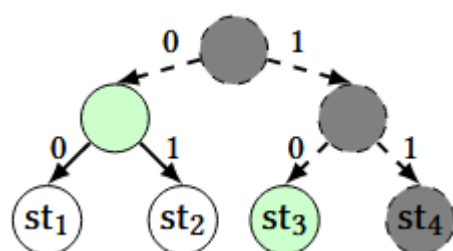
$$s_c \stackrel{?}{=} g(y) \quad (\text{或 } s_c \stackrel{?}{=} \pi_{pk}(y))$$

就能判断出这 y 真的来自链上的下一个节点，还是一个**完全随机**的新值。这样，RCPRF 的“伪随机性”立马被破坏。

- 而在输出上多套一层单向哈希 H 后，输出变成了 $H(s_{c+1})$ ，即便对手知道 s_c ，也无法从 $H(s_{c+1})$ 反推出 s_{c+1} ，更没法判定它是否真的链上下一个值。这样就满足了“RCPRF 输出对手看起来就像随机数”的安全要求。

一句话总结：**加一个哈希是为了切断“内部状态→输出值”那种可逆或可预测的联系，确保在拥有部分约束密钥的情况下，任何新输出看上去都像真·随机数，达成 RCPRF 的安全定义。**

下面结合图 5（下图）中的例子来说明，GGM-树为什么在一次“批量”评估 $[1..c]$ 时，总共只做 $O(c)$ 次 PRG 调用。



图：一棵高度为 2 ($L=4$) 的 GGM 二叉树，叶子依次对应 $st_1, \dots, st_4$ 。

1. Best Range Cover (BRC) 选出哪些子树根

我们要批量恢复 $st_1, st_2, \dots, st_c$ (这里 $c=3$)，就先从根节点 s 开始，选一个最少数量的“子树根”集合，使得它们的叶子恰好覆盖编号 $1 \sim 3$ 而不越界。这个集合称作 **Best Range Cover (BRC)**。

- 在本例中，最优覆盖是：
 - 节点 s_0 (左子树根，覆盖叶子 1 和 2)
 - 叶子节点 st_3 自身 (，因为只剩它了)

因此，**约束密钥** ck_3 就只包含这两个节点的种子：

$$ck_3 = \{s_0, st_3\}.$$

2. 从子树根展开到各叶子的 PRG 调用

拿到 ck_3 后，服务器需要实际恢复出 st_1, st_2 (而 st_3 已经直接给出，无需计算)：

1. 恢复子树 [1,2]

- 输入：子树根的种子 s_0 。
- 调用一次 PRG：

$$(s_{00} \parallel s_{01}) \leftarrow \text{PRG}(s_0).$$

这里约定：生成的左右两半正好对应叶子 1 和 2 的种子。

- 结果：

$$st_1 = s_{00}, \quad st_2 = s_{01}.$$

2. 恢复叶子 3

- st_3 已经包含在 ck_3 里，无额外代价。

一共执行了 **1 次 PRG 调用**，就得到了 $\{st_1, st_2, st_3\}$ 。

3. 为什么总体是 $O(c)$ 次

- 对每个被 BRC 选中的“子树”——假设它覆盖 m 个叶子——你需要对它的每个内部节点做一次 PRG，以把父节点的种子“分裂”成它的两个孩子。
- 整个 BRC 会拆出若干不相交的子树，这些子树的叶子总数正好是 c 。
- 一棵完全二叉子树有 $m-1$ 个内部节点，所以恢复 m 片叶子要做 $m-1$ 次 PRG。
- 把所有子树加起来，总调用次数

$$\sum_{\text{each subtree}} (m_i - 1) = \left(\sum_i m_i \right) - \#\{\text{subtrees}\} = c - O(\log L) = O(c).$$

(这里 $\#\{\text{subtrees}\} = O(\log L)$ ，而 $\log L$ 相较于 c 可忽略。)

小结

- **BRC** 保证我们只选最少的、完整覆盖前 c 叶子的子树根；
- 每个子树根展开到它所覆盖的叶子，需要做的 PRG 调用数正好等于“子树叶子数减 1”；
- 合并所有子树、恢复所有 c 个叶子，总调用次数就是 $\sum (m_i - 1) = c - O(\log L) = O(c)$ 。

因此，通过“BRC+子树展开”的策略，GGM-RCPRF 在一次批量评估 $[1..c]$ 时，恰好做 $O(c)$ 次 PRG 调用。

三种 RCPRF 构造在“可支持的输入域”大小上的区别：

- **哈希链 (hash-based) 和 GGM 树 (GGM-based) RCPRF**

这两者都基于对“第 $1..L$ ”个位置预先做结构化（哈希链要先固定链长 L ，GGM 树要先固定树的高度 $\log L$ ）。因此，它们只能在用户事先指定好的有限区间 $[1..L]$ 内生成 / 约束 PRF 输出，**无法动态扩展到超过 L 的位置**。一旦运行到 $c = L$ （哈希链用尽或者树的叶子全被覆盖），就必须重新初始化整个结构。

- **陷门置换 (TDP-based) RCPRF**

这套方案完全不依赖预先固定的链长或树高，而是利用 Trapdoor Permutation π 的可逆特性：给定任意 i ，都可以通过连续调用公钥映射 π_{pk} （或者私钥逆映射 π_{sk}^{-1} ）来生成第 i 个状态 s_i （然后再哈希得到输出）。只要保证多项式级别的运算资源，就能支持**任意大** (polynomial-size) 域上的 PRF 评估——**不需要也不存在一个上界 L** 。

为什么“无上界”很重要？

- 对于哈希链/GGM，当关键词更新频率非常高、或者系统长期运行时，很容易碰到“超出 L ”的瓶颈，届时只能停机重建。
- TDP-based 由于基于公钥置换，位置 i 只要在多项式范围内就能直接计算，不会有“用尽”的风险。

在 DISCO 中的应用

DISCO 的三种实例化——DISCO_h（哈希链）、DISCO_g（GGM 树）、DISCO_t（TDP）——各自在“效率 vs. 可用域”上做了折中：

- DISCO_{h/g}：在小 L 场景下，哈希/PRG 运算更轻量；但需要事先设定并维护一个较大的 L 。
- DISCO_t：运算上要做公钥置换（稍重），但可随意扩展到任意批次，不受预定义上界限制。

具有恒定客户端状态的DSE

之前的工作

之前的大多数动态可搜索加密（DSE）方案，它们为了实现 **前向隐私 (forward privacy)**，都采用了类似的索引结构和令牌 (token) 演化方法。具体要点如下：

1. **每个关键词维护独立的本地状态**

客户端为系统中每个关键词 w 单独维护一个可更新的状态（通常是一个计数器 a_w 或者类似的结构），这就导致客户端存储开销与关键词数量呈线性增长。

2. **更新时生成“新鲜”令牌并加密**

- 当有一条新的更新（添加或删除操作）要上传时，客户端先将对应关键词的本地状态（比如计数器）加一，然后基于该状态计算出一个新的 **秘密令牌**（secret token）。
- 客户端用这个令牌来对本次更新的 (op, id) 对进行加密，并上传给服务器。
- 由于每次都是“现用现生”的新令牌，服务器看不到不同批次或不同关键词之间的链接，从而保证了 **前向隐私**：服务器无法把一条新更新追溯到之前的任何搜索或更新请求。

3. 搜索时仅暴露历史令牌

- 当客户端要搜关键词 w 时，只把 **对应关键词在此前所有更新中使用过的那一系列令牌**（即到当前计数器值为止的令牌列表）发给服务器，而不会泄露未来还未使用到的令牌。
- 服务器拿到这批历史令牌，就能对过往上传的所有密文索引进行解密 / 比对，从而找出匹配的文件 ID；但是，它依然 **无法“预知”未来的令牌**，也就看不到之后上传的新索引，继续维护前向隐私。

4. 图 8 中的示例

如论文 Fig. 8 所示，关键词 w_1 和 w_2 在三个批次内各自演化出一系列令牌 $st(w_1)_1, st(w_1)_2, st(w_1)_3$ ，以及 $st(w_2)_1, \dots, st(w_2)_5$ 。客户端在更新时用最新令牌加密并上传，在搜索时只把到当前批次为止的令牌集交给服务器，从而满足前向隐私。

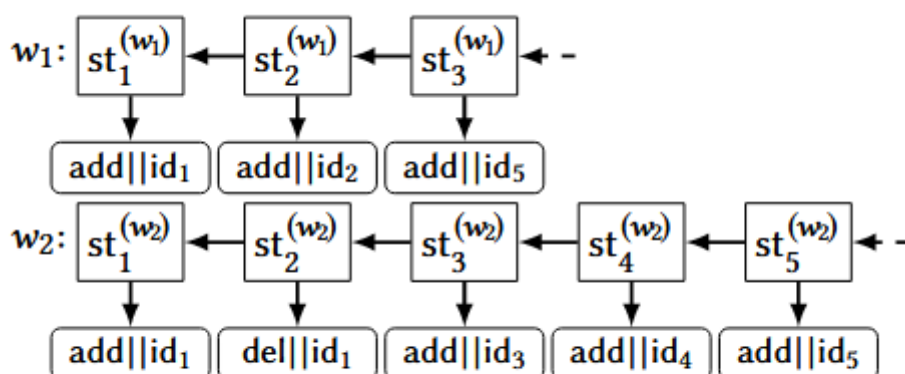


Figure 8: The index data structure using $O(|W|)$ state.

DISCO中的索引结构

1. 客户端使用一个在所有关键字之间共享的全局计数器 c ，客户端在每个更新批中增加一次 c 。
2. DISCO 重复使用关键字 w 的秘密令牌来加密批处理中与 w 关联的所有更新索引。
3. 我们采用了**可约束伪随机函数**（RCPRF）来实现对秘密令牌的演化与范围控制，并将其应用于全局计数器框架，从而彻底消除了 $O(|W|)$ 级别的客户端状态开销。

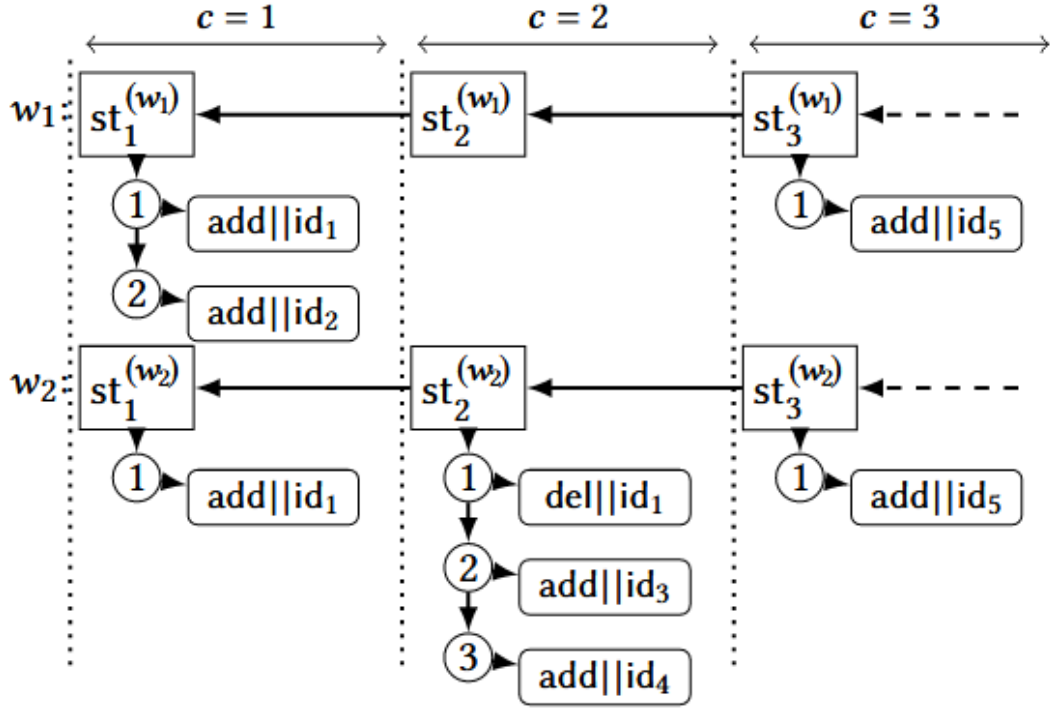


Figure 9: Index structure in DISCO using $O(1)$ state.

DISCO框架

Setup($1^\lambda; \perp$) <i>Client:</i> 1: $k_1 \leftarrow \{0, 1\}^\lambda, k_2 \leftarrow \text{SE.Gen}(1^\lambda), c \leftarrow 0$ 2: Store $\sigma \leftarrow (k_1, k_2, c)$ <i>Server:</i> 3: $\text{EDB} \leftarrow \emptyset$ Update($\sigma, \text{IN}; \text{EDB}$) <i>Client:</i> 1: Parse σ as (k_1, k_2, c) 2: Update $c \leftarrow c + 1$. 3: $\text{EDB}^{(c)} \leftarrow \emptyset$ 4: for $w \in W(\text{IN})$ do 5: $k_r^{(w)} \leftarrow F_1(k_1, w), \text{st}_c^{(w)} \leftarrow F_{rc}(k_r^{(w)}, c)$ 6: $j \leftarrow 1$ 7: for $(\text{op}, w, \text{id}) \in \text{IN}(w)$ do 8: $u \leftarrow H_1(\text{st}_c^{(w)} j)$ 9: $e \leftarrow \text{SE.Enc}(k_2, \text{op} \text{id})$ 10: $\text{EDB}^{(c)} \leftarrow \text{EDB}^{(c)} \cup \{(u, e)\}$ 11: $j \leftarrow j + 1$ 12: Send $\text{EDB}^{(c)}$ to the server	<i>Server:</i> 16: $\text{EDB} \leftarrow \text{EDB} \cup \text{EDB}^{(c)}$ Search($\sigma; \text{EDB}$) <i>Client:</i> 1: Parse σ as (k_1, k_2, c) 2: $k_r^{(w)} \leftarrow F_1(k_1, w)$ 3: $\text{ck}_c^{(w)} \leftarrow F_{rc}.\text{Constrain}(k_r^{(w)}, [c])$ 4: Send $(\text{ck}_c^{(w)}, c)$ to the server <i>Server:</i> 5: $E \leftarrow \emptyset$ 6: $\{\text{st}_i^{(w)}\}_{i \in [c]} \leftarrow F_{rc}.\text{BEval}(\text{ck}_c^{(w)}, [c])$ 7: for $i \in [c]$ do 8: $j \leftarrow 1, u \leftarrow H_1(\text{st}_i^{(w)} j)$ 9: while $\text{EDB}^{(i)}[u] \neq \perp$ do 10: $e \leftarrow \text{EDB}^{(i)}[u], E \leftarrow E \cup \{e\}$ 11: $j \leftarrow j + 1, u \leftarrow H_1(\text{st}_i^{(w)} j)$ 12: Send E to the client <i>Client:</i> 13: Decrypt E using k_2 and remove deleted identifiers to obtain $\text{DB}(w)$.
---	---

Figure 10: The pseudocode of DISCO.

用 $F_{rc}.\text{BEval}(\text{ck}_c, [x])$ 指代在 $[c]$ 范围内对 F_{rc} 进行批量评估, 输出 c 个 PRF 输出。

更新

1. 接收一个更新批次 IN

客户端将若干条文档-关键词操作 $\{[w_1 : \{(add, id_1), (add, id_2)\}]; [w_2 : \{(add, id_1)\}]\}$ 聚集成一个批次 IN, 准备一次性上传。

2. 全局计数器自增

客户端维护一个单一的全局批次计数器 c ，每处理一次新的批次就执行

```
1 |  $c \leftarrow c + 1$ 
```

3. 按关键词派生 RCPRF 主密钥

对于批次中出现的每个关键词 w ，客户端用一个常规伪随机函数 F_1 （用密钥 k_1 ）按需生成该关键词的 RCPRF 主密钥：

$$k_r^{(w)} \leftarrow F_1(k_1, w)$$

4. 生成批次令牌

有了 $k_r^{(w)}$ 和当前批次号 c ，客户端对该关键词调用 RCPRF 的约束接口得到“受限”令牌：

$$st_w^c \leftarrow \text{RCPRF}(k_r^{(w)}, c)$$

5. 批次内令牌复用 & 子计数器加密

在同一批次 c 里，如果关键词 w 出现多次更新（比如 (add, id_1) 和 (add, id_2) ），DISCO 复用同一令牌 st_w^c 并借助一个子计数器 j 来区分：

```
1 | for each occurrence  $j = 1, 2, \dots$  of  $w$  in this batch:  
2 |    $u \leftarrow H_1(st_w^c \parallel j)$   
3 |    $e \leftarrow \text{Encrypt}_{k_2}(u \parallel (op, id))$   
4 |   append  $e$  to  $\text{EDB}(c)$ 
```

这里的 H_1 是另一个哈希/PRF，用来把 st_w^c 和子计数 j 合并成唯一的索引键 u 。

6. 上传加密索引 EDB(c)

将本批次所有关键词对应的加密条目集合 $\text{EDB}(c)$ 一并上传到服务器。

查找

DISCO 的 **搜索 (Search)** 过程可以概括为以下几个步骤：

1. 客户端释放受限密钥

客户端仅需为被查询的关键词 w 生成并发布一个受限 RCPRF 密钥

$$ck_w^c = \text{RCPRF.Constrain}(k_r^{(w)}, [1..c])$$

其中 c 为当前全局批次号。该密钥能让服务器一次性恢复前 1 到 c 批次的所有令牌输出。

2. 服务器批量恢复令牌

服务器调用

$$\text{RCPRF.BEval}(ck_w^c, [1..c])$$

恢复出集合 $\{st_w^1, st_w^2, \dots, st_w^c\}$ 。

3. 分批次扫描

对于每一个批次编号 $i \in [1..c]$ ：

- 服务器用 st_w^i 和子计数器 j （在更新时同样用来区分同批次内多条更新）计算索引键

$$u = H_1(st_w^i \parallel j)$$

- 在对应的子数据库 $\text{EDB}(i)$ 中查找所有加密条目匹配该 u ，并将这些密文项收集起来。

4. 返回并解密

服务器把所有批次中匹配到的**加密条目**合并返回给客户端。客户端使用对称密钥 k_2 解密，得到若干 (op, id) 对。

5. 结果隐藏与后向隐私处理

客户端根据操作类型剔除已删除的文件标识 id ，最终构建出关键词 w 在数据库中的当前文件集合 $DB(w)$ 。整个过程中，借助“结果隐藏”（result-hiding）技术保证了 **II 型后向隐私**：即使服务器看到查询返回结果，也无法推断出被删除的或未来的更新内容。

基于哈希 RCPRF 的 DISCO

基于哈希的 RCPRF

<u>KeyGen(1^λ)</u>	<u>Constrain($k, [c]$)</u>	<u>Eval(ck_c, i)</u>
1: $s_L \xleftarrow{\$} \{0, 1\}^{2\lambda}$ 2: $k \leftarrow \{L, s_L\}$ 3: Return k	1: parse $k = (L, s_L)$ 2: $s_c \leftarrow H^{L-c}(s_L)$ 3: $ck_c \leftarrow \{L, s_c, c\}$ 4: Return ck_c	1: $ck_c = (L, s_c, c)$ 2: $s_i \leftarrow \perp, st_i \leftarrow \perp$ 3: if $i \in [c]$ then 4: $s_i \leftarrow H^{c-i}(s_c)$ 5: $st_i \leftarrow H(s_i i)$ 6: Return st_i

Figure 2: RCPRF $F_{rc}^{(h)}$ from hash chains.

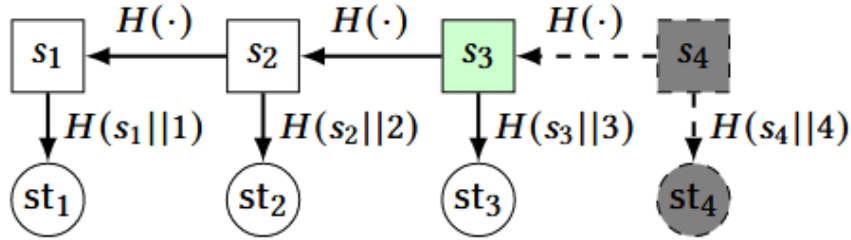


Figure 3: RCPRF $F_{rc}^{(h)}$ from hash chains. H is a hash function. The green node corresponds to a constrained key for $[1, 3]$.

包含 L 个结点的哈希链有一个最初的种子 $s_L \in \{0, 1\}^{2\lambda}$ ，其中 s_i 是第 i 个结点，由 $H(s_{i+1})$ 计算得来， s_i 是第 i 个结点， L 是哈希链的长度。 $H^n = H(\dots H(H(X)))$ 。

主密钥是 (S_L, L) ；约束算法 Constrain 从 s_L 开始，通过 H^{L-c} 计算第 c 个哈希结点，最终得到约束密钥 $ck_c \leftarrow \{L, s_c, c\}$ ；Eval 则以 s_c 为起点，向前计算 s_i 和 st_i 。

DISCO_h

对于 DISCO_h 的 $F_{rc}^{(h)}$ ，客户端直接使用 $k_r^{(w)}$ 作为初始种子初始化哈希链用于生成秘密部分

基于GGM RCPRF 的 DISCO

基于 GGM 的 RCPRF

$\text{KeyGen}(1^\lambda)$	$\text{Constrain}(k, [c])$	$\text{Eval}(\text{ck}_c, i)$
1: $s \xleftarrow{\$} \{0, 1\}^\lambda$ 2: $k \leftarrow \{L, s\}$ 3: Return k	1: $\text{ck}_c \leftarrow F_G.\text{BRC}(s, [c])$ 2: Return ck_c	1: $\text{st}_i \leftarrow \perp$ 2: if $i \in [c]$ then 3: $\text{st}_i \leftarrow F_G(\text{ck}_c, i - 1)$ 4: Return st_i

Figure 4: RCPRF $F_{\text{rc}}^{(g)}$ from GGM PRF.

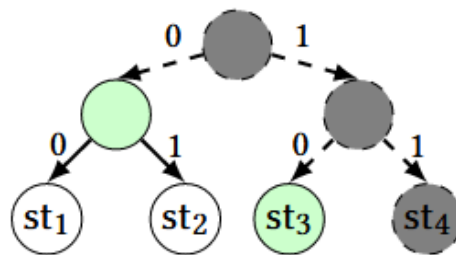


Figure 5: RCPRF $F_{\text{rc}}^{(g)}$ from GGM trees. The green nodes correspond to a constrained key for the range $[1, 3]$.

作者定义 (L, k_G) 为主密钥，计算对应范围 $[c]$ 的最佳范围覆盖作为约束密钥，对应于范围 $[c]$ 的 BRC 足以恢复 $i \in [c]$ 的所有叶节点 st_i

DISCO_g

对于 DISCO 的 $F_{\text{rc}}^{(g)}$ ，客户端直接使用 $k_r^{(w)}$ 作为初始种子（GGM Tree 的根节点）初始化 GGM Tree

复杂度与优势

- **客户端计算**：每次更新/搜索只需生成或处理 $O(\log L)$ 个 BRC 节点，对应 $O(\log L)$ 次 PRG 调用，较哈希链的 $O(L)$ 或 $O(c)$ 哈希计算显著更优。
- **客户端存储**：仍然是一个根种子 k_1 、一个对称密钥 k_2 和一个全局计数器 c ，总量 $O(1)$ 。
- **通信开销**：因揭示令牌时要传送 $O(\log L)$ 个 BRC 节点，客户端 $\rightarrow$ 服务器通信略高于哈希链方案，但当 L 较大时，这点额外开销相比计算节省是完全值得的。

基于陷门置换 RCPRF 的 DISCO

基于陷门置换的 RCPRF

<u>KeyGen(1^λ)</u>	<u>Constrain($k, [c]$)</u>	<u>Eval(ck_c, i)</u>
1: $s_1 \xleftarrow{\$} \{0, 1\}^\lambda$ 2: $(pk, sk) \leftarrow \pi.Gen(1^\lambda)$ 3: $k \leftarrow \{pk, sk, s_1\}$ 4: Return k	1: $s_c \leftarrow \pi_{sk}^{-(c-1)}(s_1)$ 2: $ck_c \leftarrow \{pk, s_c, c\}$ 3: Return ck_c	1: $ck_c = \{pk, s_c, c\}$ 2: $s_i \leftarrow \perp, st_i \leftarrow \perp$ 3: if $i \in [c]$ then 4: $s_i \leftarrow \pi_{pk}^{c-i}(s_c)$ 5: $st_i \leftarrow H(s_i i)$ 6: Return st_i

Figure 6: RCPRF $F_{rc}^{(t)}$ from trapdoor permutation.

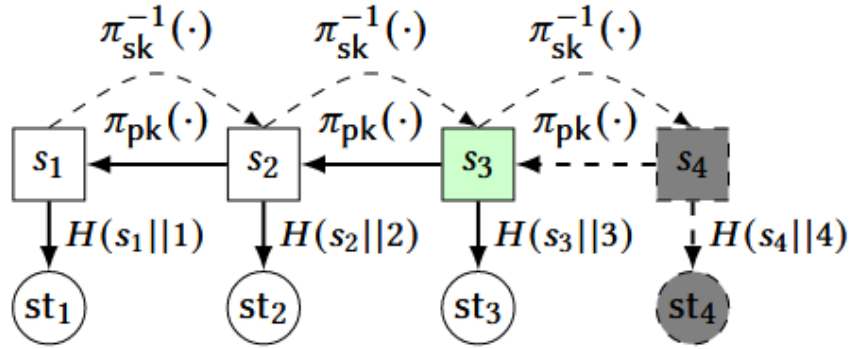


Figure 7: RCPRF $F_{rc}^{(t)}$ from a trapdoor permutation π , and H is a hash function. The green node corresponds to the constrained key for the range $[1, 3]$.

基于陷门置换的 RCPRF 保持一个链表的结构，约束算法 Constrain 从 s_1 开始，通过 $\pi_{sk}^{-(c-1)}$ 计算第 c 个结点，最终得到约束密钥 $ck_c \leftarrow \{pk, s_c, c\}$ ；Eval 则以 s_c 为起点，通过 π_{pk}^{c-i} 向前计算 s_i 和 st_i 。

DISCO_t

对于 DISCO 的 $F_{rc}^{(t)}$ ，客户端直接使用 $k_r^{(w)}$ 来确定起始种子 $s_1^{(w)}$ 。

特性与权衡

- **无限域支持**

由于映射可以任意多次进行，只要运算资源允许，就能支持“多项式大小”的批次数 c ，无需预设上界 L ，彻底消除了哈希链和 GGM 树的“用尽”风险。

- **计算开销**

每次增量映射或逆向映射都要做一次公钥置换运算，理论上比哈希/PRG 更重，但在许多 SSE 场景中，瓶颈在于 I/O 和网络；实验表明，TDP-RCPRF 的额外开销往往可以被系统整体并行或异步掩盖，不会显著影响吞吐。

- **客户端存储**

依旧是 $O(1)$ ：仅需存全局对称密钥、TDP 私钥和一个计数器。

安全性分析

DISCO实现了前向隐私和 II 型后向隐私。前向安全的实现是因为 RCPRF 的安全性，使得服务器无法通过约束的密钥计算未来的 token；由于搜索的结果使用 PCPA 安全对称密钥进行加密，只有客户端才可以解密，因此能够实现 II 型后向安全。

这种结果隐藏特性是取得 II 型 BP 的标准技术。

DISCO⁺: 优化的 DISCO

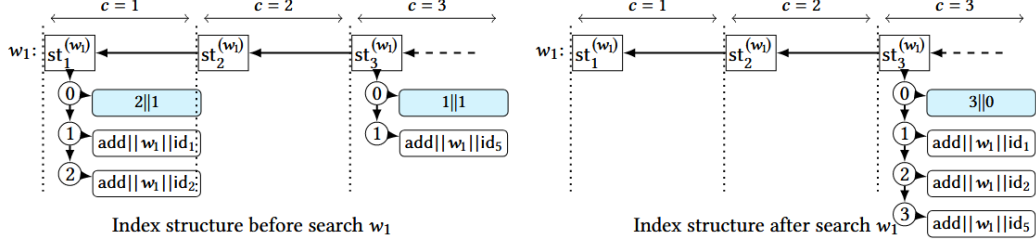


Figure 11: The (sketched) index data structure in DISCO⁺. The structure and information are reorganized after searching w_1 .

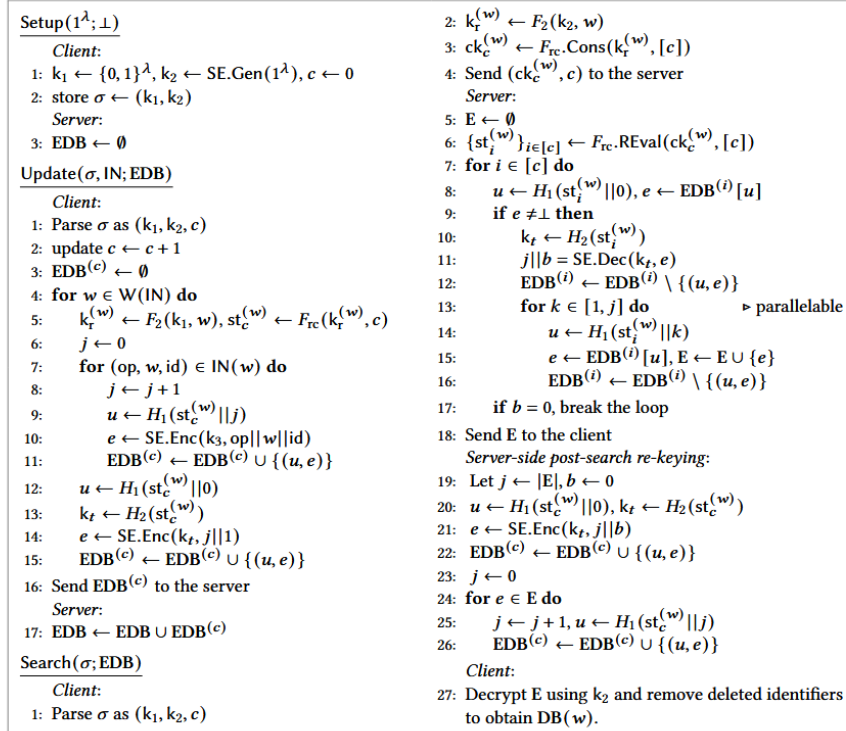


Figure 12: The pseudocode of DISCO⁺. Here H_2 is for generating a key k_t used for encrypting counter information. To make the encrypted counter ciphertext indistinguishable from normal indexes from size, padding is done whenever necessary.

因为 DISCO 的计算开销太大了，所以我们要优化一下。

启用并行搜索

在 DISCO⁺ 中，为了让服务器在搜索前就“知道”每个关键词在当前批次里到底有多少条匹配，从而启动多线程并行扫描，作者对原 DISCO 的 **Update** 协议做了如下增强：

1. 在索引条目里加密批内计数

- 令 $j = |\text{IN}(w)|$ 表示关键词 w 在本批次更新中的出现次数。
- 在原来计算并加密索引键 $u = H_1(\text{st}_c^{(w)} || j)$ 的基础上，客户端再将这个批内总计数 j 一并加密进每条索引记录里（见论文 Update 步骤第 12-15 行）。

- 注意这部分计数 j 的解密也依赖于 st_w^c ，在搜索阶段服务器拿不到它之前，是无法得知任何关于批内规模的信息的。

2. 搜索时先恢复批内计数

- 当客户端发布受限密钥 ck_w^c 并服务器解出 st_w^c 后，服务器首先用它去解密条目中隐藏的 j ，从而**获知本批次里关键词 w 有多少条加密索引**。
- 一旦知道了这个整数 j ，服务器就可以根据自身负载和硬件资源，**预先分配 t 条线程**（例如 $t = \min(j, \text{CPU核数})$ ），并将对各子数据库的扫描任务切分给这些线程并行执行。

3. 并行扫描与合并

- 多线程同时对不同子库或同一子库的不同索引区间进行匹配检查，极大提升了吞吐量，尤其当 j 很大时，可以将原本串行的 $O(j)$ 搜索时间缩短近 $1/t$ 。
- 最终各线程将找到的密文条目汇总，统一返回给客户端。

为何安全？

- 在更新时，这个批内计数 j 被加密并“绑定”到了 st_w^c 上——只有在搜到真正的 st_w^c 后，服务器才能解出它。
- 因此，DISCO⁺ 保持了与 DISCO 相同的**前向隐私与后向隐私**保证：在查询前，服务器既不知道未来批次的信息，也无法在没有合法令牌的情况下提取任何批内统计。

通过这种“先知道匹配规模、再并行扫描”的简单优化，DISCO⁺ 在**大结果集**的场景下，能够获得显著的检索加速，而不影响整个方案的安全性。

减少RCPRF 的计算

在 DISCO⁺ 中，为了避免每次对同一关键词都要从批次 1 一直评估到当前批次 c （即执行 $O(c)$ 次 RCPRF），作者设计了**后搜索重键（post-search rekeying）**机制，让服务器在一次搜索后就不再需要评估过去的老批次，从而大幅降低后续同关键词的检索开销。其思路和流程大致如下：

1. 搜索后重键（Post-Search Rekeying）

1. 利用最新令牌重加密

- 当服务器完成对关键词 w 的一次搜索后，手中已经恢复出了最新的伪随机令牌 st_w^c 。
- 服务器用这个最新令牌重新为所有匹配到的加密索引条目（集合 $E \setminus \text{mathsf{E}}$ ）生成新的“伪随机地址”：

$$u' = H_1(st_w^c \parallel j)$$

（这里的 j 是每条索引在批次内的子计数器，与更新时相同）。

- 然后把这些条目连同新地址一起**写回到最新的子数据库 $EDB(c)$** ，并**删除**它们在旧子库中的所有对应记录。

2. 删除旧条目，节省存储

- 重键完成后，所有与 w 相关的索引都只存在于最新批次的子库，不再分散在 $EDB(1), \dots, EDB(c-1)$ 里。
 - 这样服务器就不需要下次再去扫描旧子库，减少了存储压力和检索范围。
-

2. 何时停止逆推 (Stop Bit)

为了让服务器知道“已经把所有历史索引重合并到最新批次了，后面无需再评估旧批次”，DISCO⁺ 在每条加密索引里再携带一个加密的位 $b \in \{0, 1\}$ ：

- **更新时**：客户端对每个关键词 w 初次上传时，设置 $b=1$ （意味着“尚未搜索过”）。
- **搜索后**：服务器在重键阶段把 b 更新为 0 并重新加到所有重加密后的条目里。

在后续对同一关键词的搜索过程中，服务器解密出这个位 b ：

- 如果 $b=1$ ，说明还没做过一次重键，服务器才继续向前逆推评估更早的 RCPRF 状态；
- 如果 $b=0$ ，说明所有老批次都已重键到当前批次，服务器**可以立即停止**，不再执行任何逆向映射或子库扫描。

3. 效果与收益

- **首次搜索**：仍需 $O(c)$ 次 RCPRF 逆推和扫描，但完成后所有旧索引都被“压缩”到最新批次。
- **后续搜索**：由于所有索引都已重键到批次 c ，且 $b=0$ 指示“无需再往前”，服务器只需对最新批次做**一次简单扫描** ($O(j)$ 次对称解密/比较)，完全跳过 RCPRF 评估。
- 因此，对于**查询频繁**的关键词，除了第一次，后续几乎不再产生 RCPRF 代价，将检索成本降至最低。

通过“重键 + 停止位”二合一的设计，DISCO⁺ 在保持动态可搜索加密安全性的同时，实现了针对高频热点关键词的检索加速，将 RCPRF 评估成本摊薄到几乎为零。

启用数据库合并

在 DISCO⁺ 中，当应用场景下**每次更新批次都很小**（例如几十条记录），如果一直保留大量微小的子数据库，会导致服务器在每次搜索时仍需扫描很多次、执行多次 RCPRF。为此，DISCO⁺ 提供了一个“数据库合并（rebuilding）”的**客户端主动优化策略**：

1. 周期性下载与本地解密

- 客户端定期（如每隔 N 个批次）从服务器下载一段连续的子数据库集合：

$$\text{EDB}(c-N+1), \text{EDB}(c-N+2), \dots, \text{EDB}(c).$$

- 为了让客户端能够**完全恢复**每条加密条目对应的关键词 w ，在原有索引加密格式中，**额外将关键词 w 一并加密进每条记录**（见 Update 步骤第 10 行）。

2. 本地合并与剔除删除项

- 客户端用对称密钥 k_2 解密这些子数据库中的所有 $(u||w||op||id)$ 记录，重构出事务序列。
- 对序列按照关键词 w 分组，合并重复添加/删除操作，并**丢弃已删除的文档 ID**，以最精简的形式保留当前每个关键词下的有效文档列表。

3. 重新加密为一个新批次

- 基于最新的全局计数器值 $c' = c + 1$ ，客户端为每个关键词 w 计算新的 RCPRF 令牌 $\text{st}_w^{c'}$ 。
- 按照标准 Update 过程，用 $\text{st}_w^{c'}$ 和子计数器为合并后的每条记录生成新的索引键 u' ，并重新加密成**单一批次** $\text{EDB}(c')$ 。

4. 上传并删除旧数据

- 客户端将 $\text{EDB}(c')$ 一次性上传到服务器。

- 同时指示服务器**删除**原先那 N 个小子数据库，从而保持服务器上的子数据库数量稳定、搜索周期更短。
-

效果

- 通过把 N 个微小批次合并为 1 个大批次，服务器在后续对同一关键词的搜索时，仅需扫描更少的子库，减少了不必要的 RCPRF 批量评估和子库迭代次数。
- 客户端借助“关键词内嵌”这一小改动，就能在本地**丢弃删除项**并精简索引，再用常规流程重构上传，确保一致性和安全性。

这样，DISCO⁺ 在**保持 $O(1)$ 客户端状态和不破坏安全性**的前提下，通过“数据库合并”这一可选优化，显著降低了因微小批次过多而带来的搜索与存储开销。